Scheduleistic ? Complete Build, Deploy & Maintain Guide

A practical, end-to-end manual for the Scheduleistic social-media scheduling SaaS: what it is, how it was built, how to put it live on your Hostinger VPS, how tenant white-label custom domains + automatic HTTPS work, and how to maintain it day-to-day from VS Code.

Audience: the operator/owner (you). Assumes basic comfort with a terminal.

---

1. What you have

Scheduleistic is a self-hosted, multi-tenant, white-label social-media scheduler (a Buffer / Hootsuite / Postiz-style product you own outright). Core features:

? Multi-tenant organizations (agencies) ? each owns client workspaces.
? 13 social networks: LinkedIn (personal + company), Facebook, Instagram, Google Business, Pinterest, Threads, TikTok, YouTube, Mastodon, Bluesky, Medium, WordPress.
? Post composer with per-network overrides, media, scheduling, queues and recurring posts; a reliable publishing engine (queued jobs, retries, token refresh).
? Agency layer: approval workflow, client portal, comments, bulk CSV import.
? Monetization: Stripe billing (Cashier), plans + usage limits.
? White-label: per-org branding (name, colours, logo) and custom domains with automatic HTTPS.
? AI caption assistant, analytics, media auto-crop, RSS/WordPress article-to-social ingestion.
? Super-admin control plane (suspend orgs, impersonate, stats).
? Security hardening: encrypted OAuth tokens, SSRF guard, suspension enforcement, throttling, audit logging.

Tech stack

? Backend: Laravel 13 (PHP 8.4), Jetstream (auth/teams), Sanctum, Cashier.
? Frontend: Inertia.js + Vue 3 + Tailwind, built with Vite.
? Data: MySQL 8, Redis (queues, cache, sessions).
? Edge: Caddy 2 (automatic TLS, on-demand certs for custom domains).
? Runtime: Docker Compose ? five services: ‘caddy‘, ‘app‘, ‘worker‘, scheduler, plus mysql and redis.

How the pieces talk

```
Internet
|
v
[ Caddy ]  :80/:443  TLS termination + per-tenant certs
|
v
[ app ]  php artisan serve :8000  (web requests, Inertia/Vue)
|--- [ worker ]    queue:work     publishes posts, refreshes tokens, analytics
|--- [ scheduler ] schedule:work  dispatches due posts every minute, polls feeds, verifies
domains
|
+--> [ mysql ]   durable data
+--> [ redis ]   queue / cache / session
```

---

2. How it was built (so you can build one like it)

The project was delivered in phases; each was a focused, tested pull request.
This is a good template for building any SaaS like this.

Phase 0 ? Foundation

Laravel + Jetstream (teams = organizations), Docker Compose dev stack, CI
(GitHub Actions running the test suite), base models and tenancy.

Phase 1 ? Publishing engine

A SocialProvider contract + a ProviderManager registry. Each network is a
driver implementing connect (OAuth), publish(), token refresh, rate limits.
A PostComposer turns composer input into a Post + per-channel PostTargets.
Queued PublishPostJobs do the actual posting with retries.

Phase 2 ? Agency layer

Approval workflow (draft ? pending ? approved/published), a client portal
(external users assigned to a workspace), comments, queue time-slots, recurring
posts, bulk CSV import, notifications.

Phase 3 ? Monetization + white-label

Stripe via Laravel Cashier with the organization as the billable entity;
plans (config/plans.php) + UsageService enforcing limits; per-org branding;
custom-domain storage; a super-admin panel.

Phase 4 ? Advanced

8 more network drivers, AI caption assistant (pluggable LLM), analytics
(metrics capture + dashboard), media auto-crop specs, RSS/WordPress ingestion.

Phase 5 ? Security hardening

Full IDOR/authorization audit, SSRF guard, suspension enforcement, AI throttling, impersonation audit logging, mass-assignment defense.

Phase 6 ? Custom-domain TLS (this release)

Caddy on-demand TLS gated by an app /tls/check endpoint, DNS TXT domain verification, and branding-by-host so a tenant’s login page is white-labeled.

Principles worth copying

? One contract, many drivers for anything pluggable (networks, AI, payments).
? Queue everything slow (publishing, token refresh, analytics, ingestion).
? Tenant isolation in one place (a guard trait) and test it.
? Config-driven limits/plans so pricing changes need no code.
? Tests per phase + CI; never merge red.

---

3. Where to host it ? your two options

You have a Hostinger KVM 4 VPS (4 vCPU, 16 GB RAM, 200 GB NVMe, 16+ TB bandwidth) and a Business (shared) hosting plan.

Verdict: use the VPS. ?

Scheduleistic needs long-running processes (queue worker + scheduler), Docker, Redis, and the Caddy edge proxy binding ports 80/443. Shared/Business hosting cannot run any of these ? no root, no Docker, no persistent daemons, no custom ports. Business hosting is fine for a marketing landing page, not the app.

Your KVM 4 is comfortably sized: it can serve thousands of scheduled posts/day and dozens of organizations. RAM/CPU headroom is plenty (the stack idles around 1?2 GB).

> Tip: keep the Business hosting for your public marketing site
> (www.yourbrand.com) and run the app on the VPS at app.yourbrand.com.

---

4. Go-live runbook (Hostinger KVM 4, Ubuntu)

Follow top to bottom. Estimated time: 30?60 minutes (plus waiting on OAuth app approvals from each network, which can take days ? start those early).

4.1 Point DNS at the VPS

In your domain's DNS (Hostinger hPanel ? Domains ? DNS, or your registrar):

```
Type   Name   Value                TTL
A      app    <your-VPS-IPv4>      300
```

So app.yourbrand.com ? your VPS. (Add an AAAA record too if the VPS has IPv6.) Wait for it to resolve (ping app.yourbrand.com).

4.2 Prepare the server

SSH in as root (Hostinger gives you the IP + password in hPanel):

```
ssh root@<your-VPS-IPv4>
```

Create a non-root user and install Docker:

```
adduser deploy
usermod -aG sudo deploy
# Install Docker Engine + Compose plugin
curl -fsSL https://get.docker.com | sh
usermod -aG docker deploy
# Basic firewall: allow SSH + web only
ufw allow OpenSSH
ufw allow 80
ufw allow 443
ufw --force enable
```

Log back in as deploy:

```
exit
ssh deploy@<your-VPS-IPv4>
```

4.3 Get the code

```
git clone https://github.com/Shubochandrosarker/Scheduleistic-App.git scheduleistic
cd scheduleistic/scheduler/app
```

4.4 Create the ‘.env‘

```
cp .env.example .env
nano .env
```

Set at least these (full reference in the Appendix):

```
APP_NAME=Scheduleistic
APP_ENV=production
APP_DEBUG=false
APP_URL=https://app.yourbrand.com
APP_DOMAIN=app.yourbrand.com
ACME_EMAIL=you@yourbrand.com

DB_CONNECTION=mysql
DB_HOST=mysql
DB_DATABASE=scheduleistic
DB_USERNAME=scheduleistic
DB_PASSWORD=<choose-a-strong-password>

REDIS_HOST=redis
QUEUE_CONNECTION=redis
CACHE_STORE=redis
SESSION_DRIVER=redis
```

> Make the DB password here match the mysql service password in
> docker-compose.yml (change both from the default secret).

4.5 Launch the stack

```
docker compose up -d --build
```

This builds the app image (Composer + npm build happen inside), then starts Caddy, app, worker, scheduler, MySQL and Redis.

Generate the app key, run migrations, cache config:

```
docker compose exec app php artisan key:generate
docker compose exec app php artisan migrate --force
docker compose exec app php artisan config:cache
docker compose exec app php artisan route:cache
```

4.6 Create your platform-admin account

Register normally at https://app.yourbrand.com/register, then promote yourself (the flag is intentionally not web-settable):

```
docker compose exec app php artisan tinker
>>> $u = App\Models\User::where('email','you@yourbrand.com')->first();
>>> $u->forceFill(['is_platform_admin' => true])->save();
>>> exit
```

You'll now see the Admin nav item (super-admin control plane).

4.7 Verify

Visit https://app.yourbrand.com ? Caddy will have issued a Let's Encrypt certificate automatically. Log in, create a workspace, and you're live.

```
docker compose ps                 # all services "running"
docker compose logs -f caddy      # watch cert issuance
docker compose logs -f worker     # watch the publishing engine
```

---

5. Connecting the networks, Stripe, and AI

The app ships with all drivers; each network just needs your developer credentials in .env. Create an app on each platform’s developer portal, set the OAuth redirect URL to:

```
https://app.yourbrand.com/channels/callback/{provider}
```

?replacing {provider} with linkedin, facebook, instagram, google_business, pinterest, threads, tiktok, youtube, etc.

5.1 Per-network credentials (start these early ? approvals take time)

? LinkedIn ? developer.linkedin.com ? create app ? Products: "Share on LinkedIn" + "Sign In". Copy Client ID/Secret to LINKEDIN_CLIENT_ID/SECRET.
? Meta (Facebook, Instagram, Threads) ? developers.facebook.com ? create a Business app ? add Facebook Login, Instagram Graph, Threads API. One app covers all three. Set FACEBOOK_* (and Threads uses the same Meta app).
? Google (YouTube, Business Profile) ? console.cloud.google.com ? OAuth consent screen + credentials ? enable YouTube Data API / Business Profile API.
? TikTok ? developers.tiktok.com ? app with Content Posting API.
? Pinterest ? developers.pinterest.com ? app with ‘pins:write‘.
? Mastodon / Bluesky / Medium / WordPress ? token/app-password based; users paste credentials in the connect UI (no platform app needed from you).

> You don’t need every network on day one. Connect the ones your customers want;
> the rest simply won’t appear as connectable until credentials exist.

5.2 Stripe billing

1. Create a Stripe account ? Developers ? API keys. Put the secret key in STRIPE_SECRET and publishable key in STRIPE_KEY.
2. Create three Products with recurring Prices (Pro, Agency, Scale ? Free has no Stripe price). Copy the price IDs into STRIPE_PRICE_PRO, STRIPE_PRICE_AGENCY, and STRIPE_PRICE_SCALE.
3. Add a webhook endpoint ? URL ‘https://app.yourbrand.com/stripe/webhook‘, events: customer.subscription.*, invoice.*. Copy the signing secret to STRIPE_WEBHOOK_SECRET.
4. Re-cache config: ‘docker compose exec app php artisan config:cache‘.

Plans/limits live in config/plans.php ? edit the limits there, no code needed.

5.3 AI caption assistant

Use any OpenAI-compatible endpoint (OpenRouter is easiest):

```
AI_FAKE=false
AI_ENDPOINT=https://openrouter.ai/api/v1/chat/completions
AI_API_KEY=<your-key>
AI_MODEL=openai/gpt-4o-mini
```

Leave AI_FAKE=true to demo without spending. The endpoint is throttled (20/min) to cap cost.

---

6. White-label custom domains + automatic HTTPS

This is the agency selling point: each customer can run the dashboard on their own domain (e.g. social.theiragency.com) with a valid certificate, fully branded ? no work from you per domain.

6.1 How it works (architecture)

1. The org owner enters their domain under White-label ? Custom domain. The app stores it and issues a one-time verification token.
2. The owner adds two DNS records at their registrar:
? ‘TXT _scheduleistic.social.theiragency.com = scheduleistic-verify-?‘ (proves ownership)
? ‘CNAME social.theiragency.com ? app.yourbrand.com‘ (routes traffic to you)
3. The scheduler runs ‘domains:verify‘ every 5 minutes (or the owner clicks Verify now); a matching TXT record flips the domain to *verified*.
4. When a browser hits ‘https://social.theiragency.com‘, Caddy sees an unknown hostname and asks the app GET /tls/check?domain=?. The app approves only verified tenant domains (and your own). Caddy then obtains a Let’s Encrypt certificate on demand and caches it ? fully automatic.
5. ‘ResolveTenantDomain‘ middleware maps the host ? organization and serves that org’s branding, even on the guest login page.

The /tls/check gate is important: it stops attackers from making Caddy request certificates for random hostnames and hitting ACME rate limits.

6.2 What you tell your customers

> "Add a TXT record _scheduleistic.<your-domain> with the value we show you,
> and a CNAME from <your-domain> to app.yourbrand.com. Click Verify. HTTPS
> turns on automatically within a couple of minutes."

6.3 Nothing to do per-domain on the server

Caddy + the app handle issuance, renewal and routing. Certs persist in the caddy-data Docker volume (don’t delete it).

---

7. Maintaining the app from VS Code (Remote-SSH)

The cleanest way to manage a live VPS app: edit and run commands on the server through VS Code, as if it were local.

7.1 One-time setup

1. Install VS Code locally + the Remote - SSH extension (by Microsoft).
2. (Recommended) set up an SSH key so you don't type passwords:
ssh-keygen -t ed25519 # on your laptop, accept defaults
ssh-copy-id deploy@<your-VPS-IP> # upload the key
3. In VS Code: press 'F1' ? "Remote-SSH: Connect to Host?" ? deploy@<your-VPS-IP>. A new window opens running *on the VPS*.
4. File ? Open Folder ? '/home/deploy/scheduleistic'. You now edit live files with full IntelliSense, and the integrated terminal is the server.

7.2 Recommended VS Code extensions (installed into the remote)

? PHP Intelephense (PHP language features)
? Laravel Extension Pack / Laravel Blade
? Vue - Official (Volar)
? Tailwind CSS IntelliSense
? DotENV

7.3 The normal change ? deploy loop

Best practice is to commit changes and pull them on the server (not edit blind):

```
# on the VPS terminal (inside VS Code)
cd ~/scheduleistic
git pull origin main                    # get the latest code
cd scheduler/app
docker compose up -d --build            # rebuild the app image
docker compose exec app php artisan migrate --force
docker compose exec app php artisan config:cache
docker compose exec app php artisan queue:restart   # workers pick up new code
```

> Make changes locally, push to GitHub, then git pull + rebuild on the VPS.
> Avoid editing files directly in the running container ? they vanish on rebuild.

7.4 Handy day-to-day commands

```
docker compose ps                       # service health
docker compose logs -f app              # web logs
docker compose logs -f worker           # publishing engine
docker compose exec app php artisan tinker      # REPL
docker compose exec app php artisan schedule:list
docker compose exec app php artisan queue:failed # failed jobs
docker compose restart worker scheduler
```

---

8. Operations: backups, updates, monitoring, scaling

8.1 Backups (do this!)

Database (nightly cron on the VPS):

```
# crontab -e  (as deploy)
0 3 * * * docker compose -f ~/scheduleistic/scheduler/app/docker-compose.yml \
exec -T mysql mysqldump -usocialistic -p<DB_PASSWORD> scheduleistic \
| gzip > ~/backups/db-$(date +\%F).sql.gz
```

Also back up these Docker volumes (contain state you can't regenerate):
? 'mysql-data' (database)
? 'caddy-data' (issued TLS certificates)
? the '.env' file (secrets) ? store a copy in a password manager.

Pull backups off the box periodically (e.g. scp to your laptop or object storage).

8.2 Updating the app

```
cd ~/scheduleistic && git pull origin main
cd scheduler/app
docker compose up -d --build
docker compose exec app php artisan migrate --force
docker compose exec app php artisan config:cache && \
docker compose exec app php artisan route:cache
docker compose exec app php artisan queue:restart
```

8.3 Monitoring

? 'docker compose ps' + 'docker stats' for health and resource use.
? Laravel logs: 'docker compose exec app tail -f storage/logs/laravel.log'.
? The '/up' health endpoint returns 200 when the app is healthy ? point an uptime monitor (UptimeRobot, BetterStack) at https://app.yourbrand.com/up.
? Impersonation and other sensitive actions are written to the log for audit.

8.4 Scaling on the KVM 4

You have lots of headroom. When volume grows:
? Run more workers: ‘docker compose up -d --scale worker=3‘.
? Increase MySQL/Redis memory if needed (they’re modest by default).
? The single VPS handles substantial load; only consider a managed DB or a second node well past thousands of active organizations.

8.5 Troubleshooting quick table

? Cert not issued for custom domain ? check the CNAME resolves to your app domain and the domain is *verified*; docker compose logs caddy.
? Posts not publishing ? ‘docker compose logs worker‘; check queue:failed; verify the network’s OAuth token isn’t expired.
? 500 after deploy ? you probably changed ‘.env‘; run ‘config:cache‘ again; check storage/logs/laravel.log.
? Scheduler not firing ? ensure the ‘scheduler‘ service is running (schedule:work).

---

9. Appendix

9.1 ‘.env‘ reference (key settings)

```
APP_NAME, APP_ENV=production, APP_DEBUG=false, APP_KEY (php artisan key:generate)
APP_URL=https://app.yourbrand.com
APP_DOMAIN=app.yourbrand.com         # Caddy: platform domain (eager cert)
ACME_EMAIL=you@yourbrand.com         # Let’s Encrypt registration

DB_CONNECTION=mysql, DB_HOST=mysql, DB_DATABASE, DB_USERNAME, DB_PASSWORD
REDIS_HOST=redis
QUEUE_CONNECTION=redis, CACHE_STORE=redis, SESSION_DRIVER=redis

# Billing
STRIPE_KEY, STRIPE_SECRET, STRIPE_WEBHOOK_SECRET
STRIPE_PRICE_PRO, STRIPE_PRICE_AGENCY, STRIPE_PRICE_SCALE, CASHIER_CURRENCY=usd

# AI
AI_FAKE=false, AI_ENDPOINT, AI_API_KEY, AI_MODEL

# Per-network OAuth (only what you use)
LINKEDIN_CLIENT_ID/SECRET, FACEBOOK_CLIENT_ID/SECRET, GOOGLE_CLIENT_ID/SECRET, ...
```

9.2 Scheduled jobs (run by the ‘scheduler‘ service)

```
posts:dispatch-due   every minute      # publish scheduled posts
domains:verify       every 5 minutes   # verify pending custom domains
analytics:fetch      hourly            # capture engagement metrics
feeds:poll           every 15 minutes  # ingest RSS/WordPress articles
```

9.3 Useful artisan commands

```
php artisan migrate --force
php artisan tinker
php artisan queue:work / queue:failed / queue:retry all
php artisan schedule:list
php artisan domains:verify
php artisan config:cache / route:cache / optimize:clear
```

9.4 Security checklist before launch

? [ ] ‘APP_DEBUG=false‘, strong ‘APP_KEY‘, strong DB password.
? [ ] Firewall: only 22/80/443 open; SSH key auth; disable root SSH login.
? [ ] Real Stripe webhook secret set; test a subscription in test mode first.
? [ ] Backups scheduled and verified (restore once to be sure).
? [ ] Your admin account is the only platform admin.

? [ ] Review ‘scheduler/docs/03-security.md‘ for the full posture.

9.5 Repository map

```
scheduler/app/                Laravel application
app/Social/                 provider drivers + contract
app/Services/               PostComposer, Usage, Analytics, Ai, Ssrf, DomainVerification ?
app/Http/Controllers/       web controllers (incl. Admin/)
config/plans.php            plans + limits (edit pricing here)
caddy/Caddyfile             edge proxy + on-demand TLS
docker-compose.yml          the full stack
scheduler/docs/               roadmap, security, this guide
```

Scheduleistic ? you own the code, the data, and the customer relationship.
Built to be operated by one person from a single VPS and a VS Code window.